

U.E 5.1 - Intelligence Artificielle : Machine learning

Promotion FISE 2027

ENSTA BRETAGNE
2 Rue François Verny
29200 Brest Cedex 9, France



BARROS ALVES Luiz Felipe
luiz.barros@ensta.fr

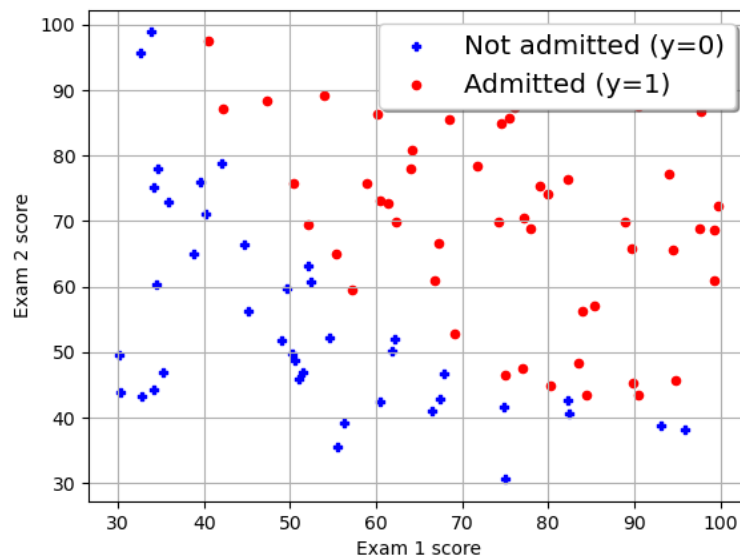
Sommaire

1	Logistic Regression	2
1.1	Visualizing the data	2
1.2	Implementation	3
1.2.1	Sigmoid function	3
1.2.2	Cost function and gradient	4
1.2.3	Learning parameters using fmin tnc	5
1.2.4	Evaluating logistic regression	6
2	Regularized logistic regression	7
2.1	Visualizing the data	7
2.2	Feature mapping	7
2.3	Cost function and gradient	7
2.3.1	Learning parameters using fmin tnc	9
2.4	Plotting the decision boundary	9
2.5	Impact of λ	10
3	Multi-class classification	11
3.1	Dataset	11
3.2	Visualizing the data	11
3.3	Vectorizing Logistic Regression	11
3.3.1	Vectorizing the cost function	11
3.3.2	Vectorizing the gradient	11
3.4	Learning a one-vs-all classifier	11
3.5	One-vs-all prediction	11
3.6	Some questions, you have to answer...	12

1 Logistic Regression

1.1 Visualizing the data

```
1 import numpy as np
2 from matplotlib import pyplot as plt
3
4
5 def plotData(X,y):
6
7     # ===== YOUR CODE HERE =====
8     # Instructions: Plot data X with different markers
9     # according the value in y
10    # =====
11
12    pos = X[(y==1).flatten(),:]
13    neg = X[(y==0).flatten(),:]
14    plt.figure()
15    plt.scatter(neg[:, 0], neg[:, 1], s=20, marker='P', color=
16                'b')
17    plt.scatter(pos[:, 0], pos[:, 1], s=20, marker='o', color=
18                'r')
19    plt.xlabel('Exam 1 score')
20    plt.ylabel('Exam 2 score')
21    plt.grid(True)
22    plt.legend(['Not admitted (y=0)', 'Admitted (y=1)'], loc='
                upper right', shadow=True, fontsize='x-large', numpoints
                =1)
23    plt.show()
24
25    # =====
```



1.2 Implementation

1.2.1 Sigmoid function

```

1 import numpy as np
2 from matplotlib import pyplot as plt
3
4
5 def plotData(X,y):
6
7     # ===== YOUR CODE HERE =====
8     # Instructions: Plot data X with different markers
9     # according the value in y
10    # =====
11
12    pos = X[(y==1).flatten(),:]
13    neg = X[(y==0).flatten(),:]
14    plt.figure()
15    plt.scatter(neg[:, 0], neg[:, 1], s=20, marker='P', color=
16                'b')
17    plt.scatter(pos[:, 0], pos[:, 1], s=20, marker='o', color=
18                'r')
19    plt.xlabel('Exam 1 score')
20    plt.ylabel('Exam 2 score')
21    plt.grid(True)
22    plt.legend(['Not admitted (y=0)', 'Admitted (y=1)'], loc='
23                upper right', shadow=True, fontsize='x-large', numpoints
24                =1)
25
26    # =====

```

1.2.2 Cost function and gradient

```
1 import numpy as np
2 from sigmoid import sigmoid
3
4 def costFunction(theta, X, y):
5     """ computes the cost of using theta as the
6         parameter for logistic regression."""
7
8     # Initialize some useful values
9     m,n = X.shape    # number of training examples and
        parameters
10    theta = theta.reshape((n,1)) # due to the use of fmin_tnc
11
12
13    # ===== YOUR CODE HERE =====
14    # Instructions: Compute the cost of a particular choice of
        theta.
15
16    #           You should set J to the cost.
17    #
18    arg1 = X @ theta
19    arg2 = sigmoid(arg1)
20    j = (-y*np.log(arg2) - (1-y)*np.log(1-arg2))
21
22    J = np.sum(j) / m
23
24    # =====
25
26    return J
```

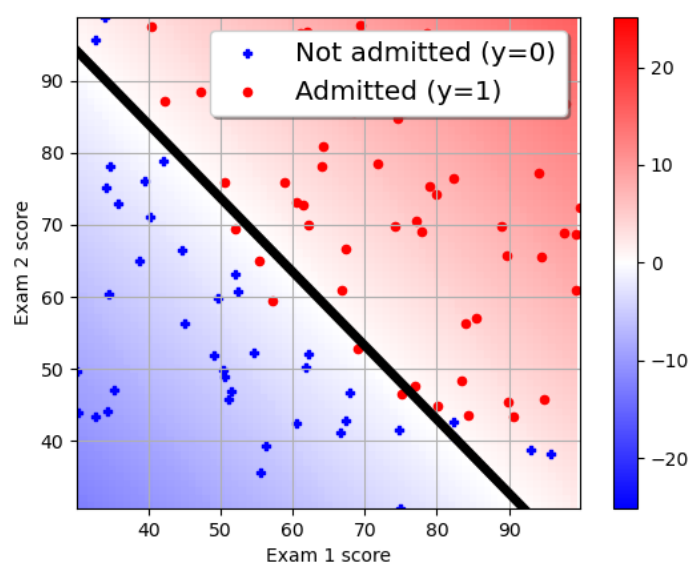
```
1 from sigmoid import sigmoid
2 import numpy as np
3
4
5 def gradientFunction(theta, X, y):
6     """
7         Compute cost and gradient for logistic regression with
            regularization
8
9         computes the cost of using theta as the parameter for
            regularized logistic
10        regression and the gradient of the cost w.r.t. to the
            parameters.
11        """
12
13    # Initialize some useful values
14    # number of training examples
15    m = X.shape[0]
16
```

```

17 # number of parameters
18 n = X.shape[1]
19 theta = theta.reshape((n,1)) # due to the use of fmin_tnc
20
21
22 # gradient variable
23 grad = 0.
24
25
26 # ===== YOUR CODE HERE =====
27 # Instructions: Compute the gradient of a particular
28 #               choice of theta.
29 #               Compute the partial derivatives and set
30 #               grad to the partial
31 #               derivatives of the cost w.r.t. each
32 #               parameter in theta
33
34 arg1 = X @ theta
35 arg2 = sigmoid(arg1)
36 grad = (X.T @ (arg2 - y)) / m
37
38 # =====
39
40 return grad

```

1.2.3 Learning parameters using fmin tnc



The function `plotDecisionBoundary.py` takes the decision at the point where the sigmoid function is equal to 0.5, which is the same as saying that the logit function is equal to 0. Then we have $\theta_0 + \theta_1 x_1 + \theta_2 x_2 = 0$, which is a linear equation in x_1 and x_2 . The function `plotDecisionBoundary.py` takes the two points where the line intersects the axes and plots the line between them.

1.2.4 Evaluating logistic regression

```

1 import numpy as np
2 from sigmoid import sigmoid
3
4
5 def predict(theta, X):
6
7     """ computes the predictions for X using a threshold at
8         0.5
9     (i.e., if sigmoid(theta'*x) >= 0.5, predict 1)
10    """
11
12    # initialisation
13    p = 0
14
15    # ===== YOUR CODE HERE =====
16    # Instructions: Complete the following code to make
17    # predictions using
18    # your learned logistic regression
19    # parameters.
20    # You should set p to a vector of 0's and 1'
21    # s
22
23    p = sigmoid(X @ theta) >= 0.5
24
25    # =====
26
27    return np.atleast_2d(p).T

```

```

Cost at theta found by scipy: 0.203498
Expected cost (approx): 0.203498

-----

theta: ['-25.1613', '0.2062', '0.2015']
Expected theta (approx): -25.1613 0.206 0.201

-----

For a student with scores 45 and 85, we predict an admission probability of 0.776291
Expected Proba (approx): 0.776291

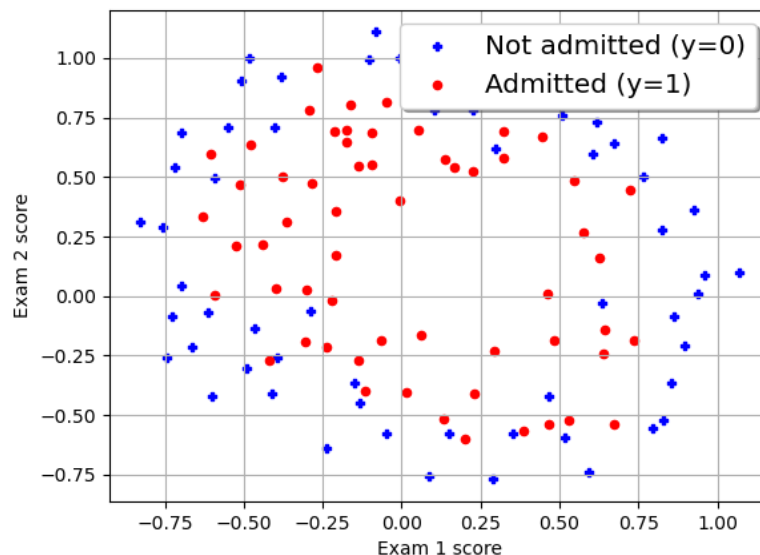
-----

Train Accuracy: 89.000000
Expected accuracy (approx): 89.0%

```

2 Regularized logistic regression

2.1 Visualizing the data



2.2 Feature mapping

2.3 Cost function and gradient

```

1 import numpy as np
2 from sigmoid import sigmoid
3
4
5 def costFunctionReg(theta, X, y, Lambda):
6     """
7     Compute cost and gradient for logistic regression with
8     regularization
9
10    computes the cost of using theta as the parameter for
11    regularized logistic regression and the
12    gradient of the cost w.r.t. to the parameters.
13    """
14    # Initialize some useful values
15    m,n = X.shape    # number of training examples and
16                     # parameters
17    theta = theta.reshape((n,1)) # due to the use of fmin_tnc
18
19    J = 0.
20
21    # ===== YOUR CODE HERE =====

```

```

19 # Instructions: Compute the cost of a particular choice of
    #       theta.
20 #               You should set J to the cost.
21 #               Compute the partial derivatives and set
    #       grad to the partial
22 #               derivatives of the cost w.r.t. each
    #       parameter in theta
23
24 # =====
25
26 arg1 = (X@theta)
27 arg2 = sigmoid(arg1)
28 J = (-1/m) * (y.T @ np.log(arg2) + (1-y).T @ np.log(1-arg2
    )) + (Lambda/(2*m))*np.sum(theta[1:]**2)
29
30 # =====
31
32 return J

```

```

1 import numpy as np
2 from sigmoid import sigmoid
3
4
5 def gradientFunctionReg(theta, X, y, Lambda):
6     """
7     Compute cost and gradient for logistic regression with
        regularization
8
9     computes the cost of using theta as the parameter for
        regularized logistic regression and the
10    gradient of the cost w.r.t. to the parameters.
11    """
12
13    # Initialize some useful values
14    m,n = X.shape # number of training examples and
        parameters
15    theta = theta.reshape((n,1)) # due to the use of fmin_tnc
16
17    grad = 0.
18
19    # ===== YOUR CODE HERE =====
20    # Instructions: Compute the gradient of a particular
        choice of theta.
21    #               Compute the partial derivatives and set
        grad to the partial
22    #               derivatives of the cost w.r.t. each
        parameter in theta
23    # =====
24

```

```

25     arg1 = (X@theta)
26     arg2 = sigmoid(arg1)
27     grad = (1/m) * (X.T @ (arg2 - y)) + (Lambda/m) * np.vstack
           ([[0], theta[1:]])
28
29
30     # =====
31
32     return grad

```

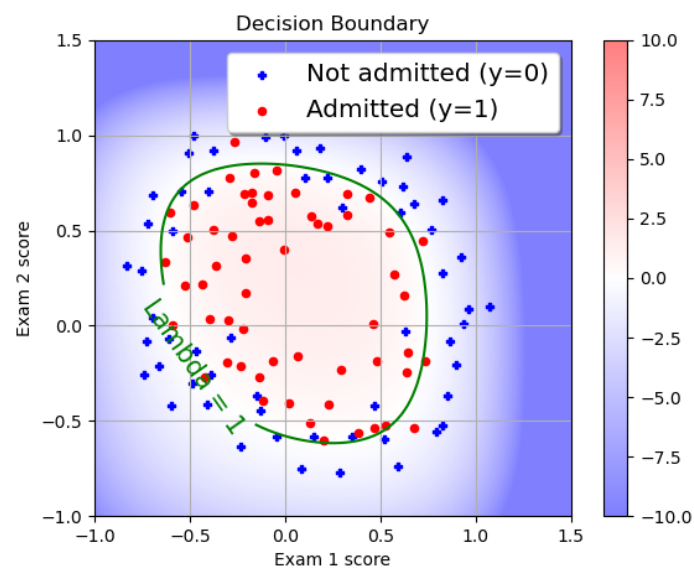
2.3.1 Learning parameters using fmin tnc

```

Cost at theta found by fmin.tnc: 0.529083
Expected cost (approx): 0.5298
.....
theta: [1.2727, 0.6253, 1.1811, -2.0199, -0.9174, -1.4317, 0.1239, -0.3655, -0.3573, -0.1752, -1.4582, -0.6518, -0.6156, -0.2747, -1.1927, -0.2422, -0.
2060, -0.0447, -0.2778, -0.2954, -0.4565, -1.0432, 0.0278, -0.2924, 0.0196, -0.3274, -0.1439, -0.9247]
Expected theta (approx): 1.27 0.63 1.18 -2.02 -0.92

```

2.4 Plotting the decision boundary



2.5 Impact of λ

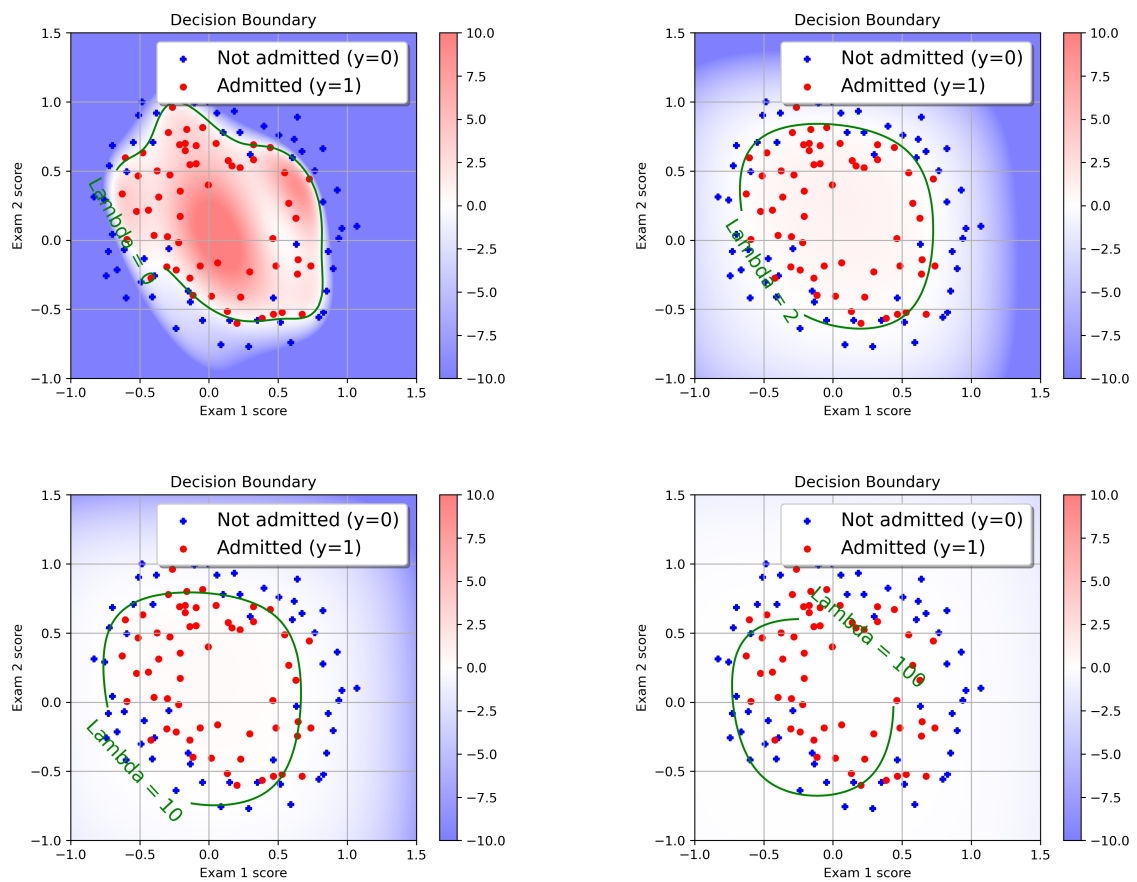
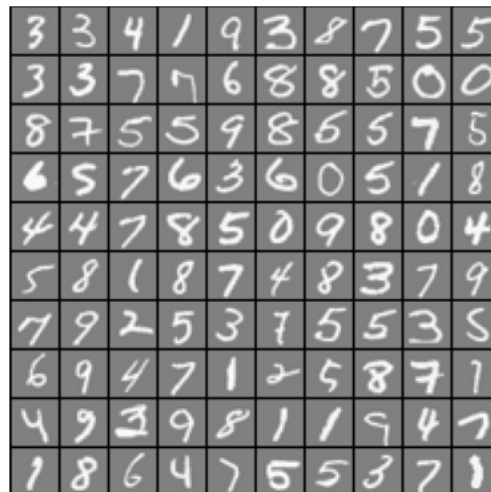


Figura 1: Évolution de la frontière de décision selon la valeur de Lambda.

3 Multi-class classification

3.1 Dataset

3.2 Visualizing the data



3.3 Vectorizing Logistic Regression

3.3.1 Vectorizing the cost function

The code is described in the subsection 2.3.

3.3.2 Vectorizing the gradient

The code is described in the subsection 2.3 as well.

3.4 Learning a one-vs-all classifier

3.5 One-vs-all prediction

```
1 import numpy as np
2
3 from sigmoid import sigmoid
4
5 def predictOneVsAll(all_theta, X):
6
7     """will return a vector of predictions
8     for each example in the matrix X. Note that X contains the
9     examples in
```

```

9      rows. all_theta is a matrix where the i-th row is a
      trained logistic
10     regression theta vector for the i-th class. You should set
      p to a vector
11     of values from 1..K (e.g., p = [1 3 1 2] predicts classes
      1, 3, 1, 2
12     for 4 examples) """
13
14 # ===== YOUR CODE HERE =====
15 # Instructions: Complete the following code to make
      predictions using
16 #         your learned logistic regression parameters (
      one-vs-all).
17 #         You should set p to a vector of predictions (
      from 1 to
18 #         num_labels).
19 #
20 # Hint: This code can be done all vectorized using the max
      function.
21 #         In particular, the np.argmax function can return the
      index of the max
22 #         element, for more information see 'numpy.argmax' on
      the numpy website.
23 #         If your examples are in rows, then, you can use
24 #         np.argmax(probs, axis=1) to obtain the max for each
      row.
25 #
26
27     p = np.argmax(sigmoid(X @ all_theta.T), axis=1)
28
29
30
31 # =====
32
33     return np.atleast_2d(p).T

```

3.6 Some questions, you have to answer...

1. Identifiez les différences d'approches entre le TP1 et le TP2 et les différences de résolution du problème.

Dans la TP1, on a fait de la régression linéaire. Le but était de deviner une valeur exacte et continue comme le prix d'une maison. Dans le TP2, on a fait de la classification. Le but n'était plus de trouver un chiffre mais de classer des données dans des catégories, comme dire "oui" ou "non" (admis ou refusé). Dans le TP1, on a écrit nous mêmes tout le code pour faire avancer l'ordinateur pas à pas et trouver l'erreur minimum. Dans le TP2 on a utilisé une fonction Python déjà prête la `fmin_tnc` qui fait tout ce travail d'optimisation toute

seule. On a juste eu à lui donner la fonction pour calculer l'erreur (le coût) et le gradient.

2. A quoi sert le principe de régularisation ?

La régularisation sert à empêcher le modèle d'être trop complexe. Quand on donne beaucoup d'informations à l'ordinateur il a tendance à vouloir apprendre les données d'entraînement par cœur, avec tous leurs petits défauts. La régularisation le force à garder des calculs plus simples pour qu'il reste capable de bien juger de nouvelles données qu'il n'a jamais vues.

3. Décrivez le problème du sur-apprentissage et comment on y répond dans ce TP. Quelle est l'influence de la valeur de λ ?

Le sur-apprentissage (ou overfitting), c'est quand l'algorithme devient trop perfectionniste. Il dessine une frontière complètement tordue pour passer exactement sur chaque point d'entraînement. La solution dans le TP on utilise la régularisation pour pénaliser l'ordinateur s'il fait des courbes trop compliquées. C'était possible de remarquer que si λ est trop petit, on laisse faire l'ordinateur et on a du sur-apprentissage. Si λ est trop grand (comme 100), on punit trop le modèle et il ne fait plus d'efforts (c'est le sous-apprentissage). Le but est de trouver un λ moyen pour avoir une courbe douce et naturelle.

4. Dans quelles circonstances et pour quelles raisons utilise-t-on l'approche multiclasse ? Quelles sont les différentes possibilités pour cette approche?

On l'utilise quand on a plus de deux choix possibles à la fin. La régression logistique normale ne sait répondre que par "oui" ou "non" (2 choix). Mais pour lire des numéros de codes postaux on doit deviner un chiffre entre 0 et 9 (10 choix). On est obligé d'adapter la méthode.

Une approche est un contre tous (One-vs-All). C'est ce qu'on a fait dans le TP. On entraîne le programme à séparer une seule catégorie de toutes les autres réunies par exemple : "est-ce un 3, ou est-ce que ce n'est pas un 3 ?", et on répète ça pour chaque chiffre.